

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO5: Analyze object-oriented software using quality assurance and usability testing Techniques (BL4,BL5)
Assessment Tool Weightage: 35%

QUESTIONS: 5

Total Marks:35

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

*I E. Sakthi priya (192520041) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

E. Sakthi priya

RUBRICS FOR CALCULATION:

Constraint-Based Problem Solving Assessment Rubric

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Annexure A

Constraint-Based Problem Solving

Q1. Scenario: Online Banking System

System Components:

Account Management, Fund Transfer, Bill Payment, Transaction History

Given Constraints:

- Transaction completion time < 2 seconds
- Multi-factor authentication required
- Support 5000+ concurrent users

Question:

Analyze the system using quality assurance techniques under the given constraints. Identify potential defects related to security, performance, and reliability. Propose suitable testing strategies and justify how they ensure system quality. **(BL4)**

Q2. Scenario: E-Commerce Shopping Application

System Features:

Product Search, Shopping Cart, Checkout, Payment Gateway

Usability Constraints:

- Checkout process within 3 steps
- Easy product navigation
- Responsive design for mobile and desktop devices

Question:

Apply usability testing techniques to evaluate the application. Identify usability challenges and recommend design improvements that satisfy the given constraints. Explain how the proposed changes improve customer satisfaction and efficiency. **(BL4)**

Q3. Scenario: Hospital Management System

Modules:

Patient Registration, Appointment Scheduling, Medical Records, Billing

QA Constraints:

- Patient data accuracy must be maintained
- System availability 24/7
- Simultaneous access by multiple departments

Question:

Use quality assurance practices to analyze the system. Identify critical test cases and discuss how the constraints influence testing priorities, defect prevention, and overall system reliability. **(BL5)**

Q4. Scenario: Smart Library Management System**Features:**

Book Search, Book Reservation, Borrow/Return, Notifications

Usability Constraints:

- Search results displayed within 2 seconds
- Accessible interface for all age groups
- Minimal learning curve for first-time users

Question:

Conduct a usability evaluation using appropriate testing methods. Analyze how the given constraints impact interface design and recommend improvements to enhance usability and accessibility. **(BL5)**

Q5. Scenario: Airline Reservation System**Modules:**

User Login, Flight Search, Seat Selection, Ticket Booking

QA & Usability Constraints:

- No booking duplication
- System must support peak holiday traffic
- Booking interface should be simple and error-free

Question:

Apply both quality assurance and usability testing techniques under the specified constraints. Identify major risks, propose suitable testing methods, and explain how the system can be optimized for reliability, performance, and user experience. **(BL4)**

Q1. Online Banking System

The online banking system consists of Account Management, Fund Transfer, Bill Payment, and Transaction History. The major constraints are transaction completion within 2 seconds, mandatory multi-factor authentication, and support for 5000+ concurrent users.

1. Problem Understanding

The system handles sensitive financial information and transactions. Therefore, the main quality requirements are:

- **Security** – prevent unauthorized access and fraudulent transactions.
- **Performance** – transactions must finish in less than 2 seconds.
- **Reliability** – transactions should not be lost, duplicated, or incorrectly processed.
- **Scalability** – the system must support at least 5000 concurrent users.

2. Potential Defects

Area	Potential Defect
Security	Weak password validation
Security	MFA bypass or incorrect OTP validation
Security	Unauthorized access to another user's account
Security	Session hijacking
Security	SQL injection
Performance	Transaction takes more than 2 seconds
Performance	System becomes slow with 5000+ users
Reliability	Fund transfer succeeds but balance is not updated
Reliability	Duplicate fund transfer
Reliability	Transaction history not updated
Reliability	System failure during payment

3. Testing Strategies

a. Security Testing

- Test login authentication and authorization.
- Verify that MFA is mandatory.
- Test invalid and expired OTPs.
- Perform penetration testing for SQL injection and session attacks.
- Verify that users cannot access another user's account.

b. Performance Testing

- Conduct load testing with 5000+ concurrent users.

- Measure transaction response time.
- Perform stress testing by increasing users beyond the expected load.
- Check CPU, memory, database and network performance.

c. Functional Testing

Test:

- Account creation and login.
- Fund transfer.
- Bill payment.
- Transaction history.
- Balance updates.

d. Reliability Testing

- Interrupt the system during a transaction.
- Verify that failed transactions are rolled back.
- Check that transactions are not duplicated.
- Test database recovery and backup mechanisms.

4. Proposed Solutions

- Use strong authentication with MFA.
- Apply role-based authorization.
- Use encryption for sensitive data.
- Use database transactions to ensure atomic fund transfers.
- Use load balancing and scalable servers.
- Optimize database queries and indexing.
- Implement proper logging and monitoring.

5. Justification

Security testing prevents unauthorized financial access, performance testing ensures that the system satisfies the <2 second requirement, and load/stress testing verifies support for 5000+ concurrent users. Reliability and recovery testing ensure that financial transactions remain correct even when failures occur.

Conclusion:

Combining security, functional, performance, load, stress and reliability testing helps ensure that the banking system is secure, fast, scalable and dependable.

Q2. E-Commerce Shopping Application

The application provides Product Search, Shopping Cart, Checkout and Payment Gateway. The usability constraints require checkout within 3 steps, easy navigation, and responsive operation on mobile and desktop devices.

1. Usability Testing Techniques

a. User Testing

Ask representative users to perform tasks such as:

1. Search for a product.
2. Add it to the cart.
3. Complete checkout.

Observe the time taken, errors and difficulties.

b. Task-Based Testing

Give users specific tasks and measure:

- Task completion rate.
- Completion time.
- Number of errors.
- Number of clicks/steps.

c. Think-Aloud Testing

Users explain what they are thinking while navigating the application. This helps identify confusing buttons, labels and navigation.

d. A/B Testing

Compare two versions of important screens such as the checkout page and determine which produces better task completion.

2. Usability Challenges

- Too many checkout screens.
- Difficult product categories.
- Poor search results.
- Confusing navigation menus.
- Small buttons on mobile devices.
- Forms requiring unnecessary information.
- Poor error messages.
- Payment errors without clear explanations.

3. Design Improvements

Checkout

Reduce checkout to three simple stages:

Cart → Address/Delivery → Payment & Confirmation

Use:

- Autofill.
- Guest checkout.
- Clear progress indicators.
- Large and clearly labelled buttons.

Navigation

- Use simple categories.
- Provide search suggestions.
- Add filters and sorting.
- Display breadcrumbs.

Responsive Design

- Use responsive layouts.
- Make buttons large enough for touch.
- Optimize images.
- Ensure text is readable on smaller screens.

4. Effect on Customer Satisfaction

These changes reduce the time and effort required to purchase products. Simple navigation allows customers to find products faster, while a three-step checkout reduces abandonment. Responsive design provides a consistent experience across mobile and desktop devices.

Conclusion:

Usability testing helps identify user difficulties before deployment. Simplifying checkout, improving navigation and implementing responsive design makes the application efficient, accessible and customer-friendly.

Q3. Hospital Management System

The system contains Patient Registration, Appointment Scheduling, Medical Records and Billing. The important constraints are maintaining patient data accuracy, providing 24/7 availability, and supporting simultaneous access by multiple departments.

1. Problem Understanding

A hospital system handles critical patient and billing information. Therefore, incorrect data or system downtime can seriously affect hospital operations.

The main quality priorities are:

1. Data accuracy.
2. Availability.
3. Data consistency.
4. Concurrent access.
5. Security.
6. Reliability.

2. Critical Test Cases

Module	Critical Test Case	Expected Result
Registration	Enter valid patient details	Patient record created correctly
Registration	Enter invalid/duplicate data	Error should be displayed
Appointment	Book available slot	Appointment successfully created
Appointment	Book already occupied slot	Booking should be rejected
Medical Record	Update patient record	Correct information should be stored
Medical Record	Multiple departments access record	Data remains consistent
Billing	Generate bill	Correct amount should be calculated
Billing	Payment failure	Transaction should not create incorrect bill
System	Server failure	System should recover without data loss
Concurrency	Multiple users modify data	No inconsistent data should occur

3. Quality Assurance Practices

Functional Testing

Verify every hospital module performs according to requirements.

Integration Testing

Check whether modules communicate correctly. For example:

Patient Registration → Appointment → Medical Record → Billing.

Data Validation Testing

Verify:

- Patient IDs.
- Names.
- Medical information.
- Appointment details.
- Billing information.

Concurrency Testing

Multiple departments should access patient records simultaneously without overwriting or corrupting data.

Availability Testing

Perform failure and recovery testing to verify that the system can operate continuously.

Security Testing

Ensure only authorized hospital staff can access sensitive records.

4. Influence of Constraints

Data accuracy:

Testing must prioritize validation, database consistency and transaction correctness.

24/7 availability:

Reliability, backup, recovery and failover testing become high priorities.

Simultaneous access:

Concurrency and transaction testing are essential to prevent conflicting updates.

5. Defect Prevention

Defects can be prevented through:

- Input validation.
- Database constraints.
- Access control.
- Code reviews.
- Automated testing.
- Transaction management.
- Regular backups.
- Continuous monitoring.

Conclusion:

The constraints make data accuracy, availability and concurrency testing the highest priorities. These practices improve reliability and ensure that hospital staff can safely access accurate information whenever required.

Q4. Smart Library Management System

The system includes Book Search, Book Reservation, Borrow/Return and Notifications. The constraints require search results within 2 seconds, accessibility for all age groups and a minimal learning curve for first-time users.

1. Usability Evaluation

a. User Testing

Ask first-time users to perform tasks such as:

- Search for a book.
- Reserve a book.
- Borrow a book.
- Return a book.

Observe whether they can complete tasks without assistance.

b. Task Completion Testing

Measure:

- Task completion rate.
- Time taken.
- Number of errors.
- Number of attempts.

c. Think-Aloud Testing

Users describe their thoughts while using the system. This identifies confusing menus, icons and instructions.

d. Accessibility Testing

Test the interface for users of different age groups by checking:

- Font size.
- Text readability.
- Button size.
- Colour contrast.
- Keyboard navigation.
- Clear labels.

2. Usability Challenges

- Complicated search interface.
- Small text or buttons.
- Confusing icons.
- Poor search results.
- Difficult reservation process.
- Unclear availability information.
- Too many steps for common tasks.

3. Design Improvements

Search

- Provide a simple search box.
- Add autocomplete.
- Use filters such as author, category and availability.
- Optimize database queries so results appear within 2 seconds.

Accessibility

- Use readable fonts.
- Provide sufficient contrast.
- Use large buttons.
- Use clear text labels instead of relying only on icons.
- Support keyboard navigation.

Minimal Learning Curve

- Keep the homepage simple.
- Use familiar terminology.
- Maintain consistent menus.
- Provide short instructions and helpful error messages.

4. Impact on Usability

Fast search reduces waiting time. Accessible design allows users from different age groups to use the system comfortably. A simple interface reduces the amount of training required for first-time users.

Conclusion:

User testing, task-based testing and accessibility testing can identify usability problems. A simple, fast and accessible interface improves efficiency, user satisfaction and ease of learning.

Q5. Airline Reservation System

The system contains User Login, Flight Search, Seat Selection and Ticket Booking. The major constraints are preventing booking duplication, supporting peak holiday traffic and providing a simple, error-free booking interface.

1. Major Risks

Risk	Possible Problem
Duplicate booking	Same seat booked by two users
Concurrency	Multiple users select the same seat
Performance	System becomes slow during holidays
Reliability	Booking fails after payment
Usability	Users make errors during booking
Payment	Payment succeeds but ticket is not generated
Search	Incorrect flight availability displayed

2. Quality Assurance Testing

Functional Testing

Test:

- Login.
- Flight search.
- Flight selection.
- Seat selection.
- Ticket booking.

Integration Testing

Verify communication between flight search, seat inventory, payment and ticket generation.

Concurrency Testing

Simulate many users attempting to reserve the same seat simultaneously.

The system should allow only one successful booking.

Load Testing

Simulate peak holiday traffic and verify acceptable response times.

Stress Testing

Increase the number of users beyond expected capacity to identify the system's breaking point.

Recovery Testing

Test server/network failures during booking and verify that the system does not create incomplete or duplicate reservations.

3. Usability Testing

Conduct:

- User testing.
- Task-based testing.
- Think-aloud testing.
- Accessibility testing.

Users should be able to complete:

Login → Search Flight → Select Seat → Book Ticket

without confusion.

4. Design and Technical Improvements

Prevent Duplicate Booking

- Use transaction management.
- Lock or temporarily reserve selected seats.
- Use unique booking/seat constraints.
- Confirm seat availability again before final booking.

Handle Peak Traffic

- Use load balancing.
- Scale servers according to demand.
- Optimize database queries.
- Use caching where appropriate.

Improve Interface

- Use a simple step-by-step booking process.
- Clearly display available/unavailable seats.
- Show flight details before confirmation.
- Provide clear error messages.
- Display booking confirmation immediately.

5. Optimization

Requirement	Solution
Reliability	Transactions, backups and recovery testing
Performance	Load/stress testing and server scaling

Requirement	Solution
No duplication	Concurrency testing and database constraints
Simple interface	Usability and task-based testing
Error-free booking	Validation and clear confirmation

6. Justification

QA testing ensures that the system works correctly under normal and extreme conditions. Concurrency testing specifically addresses duplicate seat booking, while load and stress testing prepare the system for holiday traffic. Usability testing reduces user errors and makes booking easier.

Conclusion:

Combining functional, integration, concurrency, load, stress, recovery and usability testing ensures that the airline reservation system is reliable, high-performing and easy to use.